

UNIT 4 MODEL EVALUATION AND DEPLOYMENT

Introduction, Model Validation, Rule Induction Using CHAID, Automating Models for Categorical and Continuous targets, Comparing and Combining Models, Evaluation Charts for Model Comparison, Meta Level Modeling.

4.1 MODEL VALIDATION AND DEPLOYMENT:

Validating the machine learning model outputs is important to ensure its accuracy. When a machine learning model is trained, a huge amount of training data is used and the main aim of checking the model validation provides an opportunity for machine learning engineers to improve the data quality and quantity. As it happens, without checking and validating the model it is not right to rely on its prediction. And in sensitive areas like healthcare and self-driven vehicles, any kind of mistake in object detection can lead to major fatalities due to wrong decisions taken by the machine in real-life predictions. And validating the ML model at the training and development stage helps to make the model make the right predictions. Some added advantages of Model Validation are as follows.

- Scalability and flexibility
- Reduce the costs
- Enhance the model quality
- Discovering more errors
- Prevents the model from overfitting and under fitting.

Machine learning model deployment is the process of placing a finished machine learning model into a live environment where it can be used for its intended purpose. Models can be deployed in a wide range of environments, and they are often integrated with apps through an API so they can be accessed by end users. Models are usually developed in an environment with carefully prepared data sets, where they are trained and tested. Most models created during the development stage do not meet desired objectives. Few models pass their test and those that do represent a sizable investment of resources. So moving a model into a dynamic environment can require a great deal of planning and preparation for the project to be successful. Before model deployment, there are a couple of criteria that machine learning model need to achieve:

- **Portability:** this refers to the ability of your software to be transferred from one machine or system to another. A portable model is one with a relatively low response time and one that can be rewritten with minimal effort.
- **Scalability:** this refers to how large your model can scale. A scalable model is one that doesn't need to be redesigned to maintain its performance.

4.2 RULE INDUCTION USING CHAID:

Outstanding amongst other supervised learning methods are tree-based algorithm. Tree-based algorithm is predictive models with higher accuracy, in simple understanding. Decision tree learning or classification Trees are a collection of divide and conquer problem-solving strategies that use tree-like structures to predict the value of an outcome variable. The tree starts with the root node consisting of the complete data and thereafter uses intelligent strategies to split the nodes into multiple branches. The original dataset divided into subsets in this process. There are different algorithms written to assemble a decision tree, which can be utilized by the problem. A few of the commonly used algorithms are listed below:

1. CART (Classification And Regression Tree)
2. ID3 (Iterative Dichotomiser 3)
3. C4.5
4. CHAID (Chi-square automatic interaction detection)

To find the most dominant feature, chi-square tests will be used in CHAID whereas ID3 uses information gain, C4.5 uses gain ratio and CART uses the GINI index. **Chi-Square** is a statistical measure to find the difference between child and parent nodes. To calculate this we find the difference between observed and expected counts of target variable for each node and the squared sum of these standardized differences will give us the Chi-square value. The formula of chi-square testing is easy.

$$= \sqrt{\frac{(Actual - Expected)^2}{Expected}}$$

Data set:

We are going to build decision rules for the following data set. The decision column is the target we would like to find based on some features. By The Way, we will ignore the day column because it just the row number. We need to find the most important feature with respect to target columns to choose the node to split data in this data set.

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Humidity feature

There are two types of the class present in humidity columns such that high and normal. Now we will calculate the chi-square values for them. For each row, the total column is the sum of yes and no decisions. Half of the total column is called Expected values because there are 2 classes in the decision. It is easy to calculate the chi-squared values based on this table.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
High	3	4	7	3.5	0.267	0.267
Normal	6	1	7	3.5	1.336	1.336

chi-square yes for high humidity is $\sqrt{((3 - 3.5)^2 / 3.5)} = 0.267$

So, the chi-square value of the humidity feature is

$$= 0.267 + 0.267 + 1.336 + 1.336$$

$$= 3.207$$

Now, we will find chi-square values for other features also. The feature having the maximum chi-square value will be the decision point.

Wind feature

There are two types of the class present in wind columns such that weak and strong. The following table is the below table.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
Weak	5	3	8	4	1	1
Strong	3	3	6	3	0	0

So, the chi-square test value of the wind feature is

$$= 0.802 + 0.802 + 0 + 0$$

$$= 1.604$$

Temperature feature

There are three types of the class present in temperature columns such that hot, cool and mild. The following table is the below table.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
Hot	2	2	4	2	0	0
Mild	4	2	6	3	0.577	0.577
Cool	3	1	4	2	0.707	0.707

So, the chi-square test value of the temperature feature is

$$= 0 + 0 + 0.577 + 0.577 + 0.707 + 0.707$$

$$= 2.569$$

Outlook feature

There are three types of a class present in temperature columns such that sunny, rain, and overcast. The following table is the below table.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
Sunny	2	3	5	2.5	0.316	0.316

Overcast	4	0	4	2	1.414	1.414
Rain	3	2	5	2.5	0.316	0.316

So, the chi-square test value of the outlook feature is

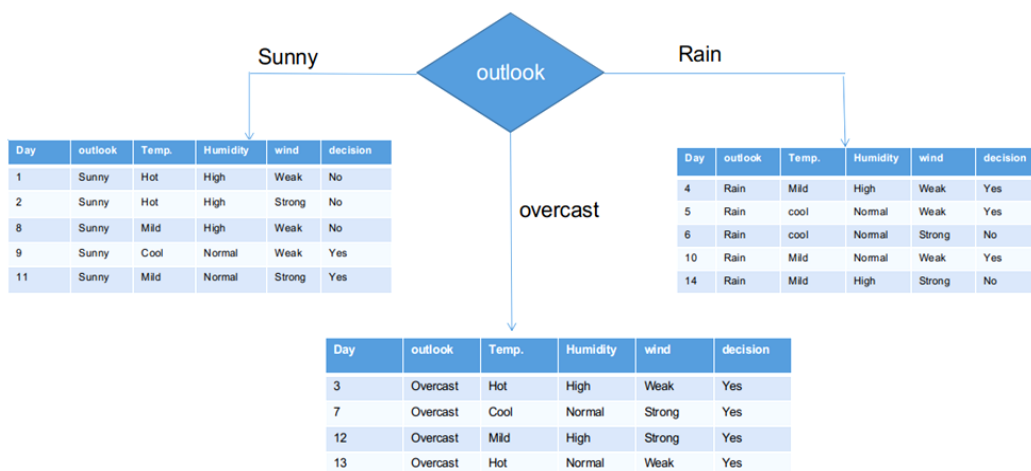
$$= 0.316 + 0.316 + 1.414 + 1.414 + 0.316 + 0.316$$

$$= 4.092$$

We have calculated the chi-square values of all features. Let's see them all at one table.

Feature	Chi-Square Value
Outlook	4.092
Temperature	2.569
Humidity	3.207
Wind	1.604

As seen, the outlook column has the most elevated and highest chi-square value. This implies that it is the main component feature. Along with these values, we will put this feature to the root node.



We've separated the raw information based on the outlook classes on the illustration above. For instance, the overcast branch simply has a yes decision in the sub informational dataset. This implies that the CHAID tree returns YES if the outlook is overcast. Both sunny and rain branches have yes and no decisions. We will apply chi-square tests for these sub informational datasets.

Outlook = Sunny branch

This branch has 5 examples. Presently, we search for the most predominant feature. By The Way, we will disregard the outlook feature now since they are

altogether the same. At the end of the day, we will find out the most predominant columns among temperature, humidity, and wind.

Day	Outlook	Temp.	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Humidity feature when the outlook is Sunny

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
High	0	3	3	1.5	1.225	1.225
Normal	2	0	2	1	1	1

Chi-square value of humidity feature for sunny outlook is

$$= 1.225 + 1.225 + 1 + 1$$

$$= 4.449$$

Wind feature when the outlook is Sunny

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
Weak	1	2	3	1.5	0.408	0.408
Strong	1	1	2	1	0	0

Chi-square value of wind feature for sunny outlook is

$$= 0.408 + 0.408 + 0 + 0$$

$$= 0.816$$

Temperature feature when the outlook is Sunny

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
Hot	0	2	2	1	1	1
Mild	1	1	2	1	0	0
Cool	1	0	1	0.5	0.707	0.707

So, the chi-square value of temperature feature for sunny outlook is

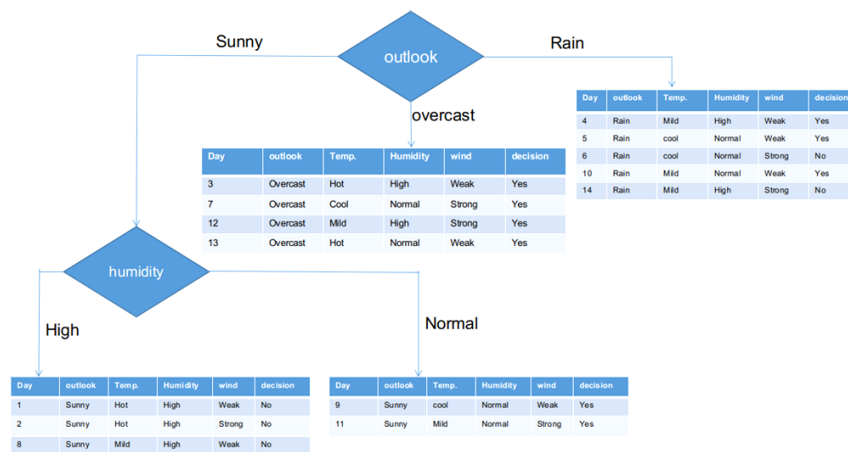
$$= 1 + 1 + 0 + 0 + 0.707 + 0.707$$

$$= 3.414$$

We have found chi-square values for sunny is outlook. Let's see them all at a table.

Feature	Chi-Square Value
Temperature	3.414
Humidity	4.449
Wind	0.816

Presently, humidity is the most predominant feature for the sunny outlook branch. We will put this feature as a decision rule.



Presently, both humidity branches for sunny outlook have only one decision as delineated previously. CHAID tree will return NO for sunny outlook and high humidity and it will return YES for sunny outlook and normal humidity.

Outlook = Rain branch

This branch actually has both yes and no decisions. We need to apply the chi-square test for this branch to find out an accurate decision. This branch has 5 distinct instances as demonstrated in the accompanying sub informational collection dataset. Now we find out the most predominant feature among temperature, humidity and wind.

Day	Outlook	Temp.	Humidity	Wind	Decision
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
10	Rain	Mild	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Wind feature for rain outlook

There are two types of a class present in wind feature for rain outlook such that weak and strong.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
Weak	3	0	3	1.5	1.225	1.225
Strong	0	2	2	1	1	1

So, the chi-square value of wind feature for rain outlook is

$$= 1.225 + 1.225 + 1 + 1$$

$$= 4.449$$

Humidity feature for rain outlook

There are two types of a class present in humidity feature for rain outlook such that high and normal.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
High	1	1	2	1	0	0
Normal	2	1	3	1.5	0.408	0.408

Chi-square value of humidity feature for rain outlook is

$$= 0 + 0 + 0.408 + 0.408$$

$$= 0.816$$

Temperature feature for rain outlook

There are two types of a class present in temperature features for rain outlook such that mild and cool.

	yes	No	Total	Expected	Chi-square Yes	Chi-square No
--	-----	----	-------	----------	----------------	---------------

Mild	2	1	3	1.5	0.408	0.408
Cool	1	1	2	1	0	0

Chi-square value of temperature feature for rain outlook is

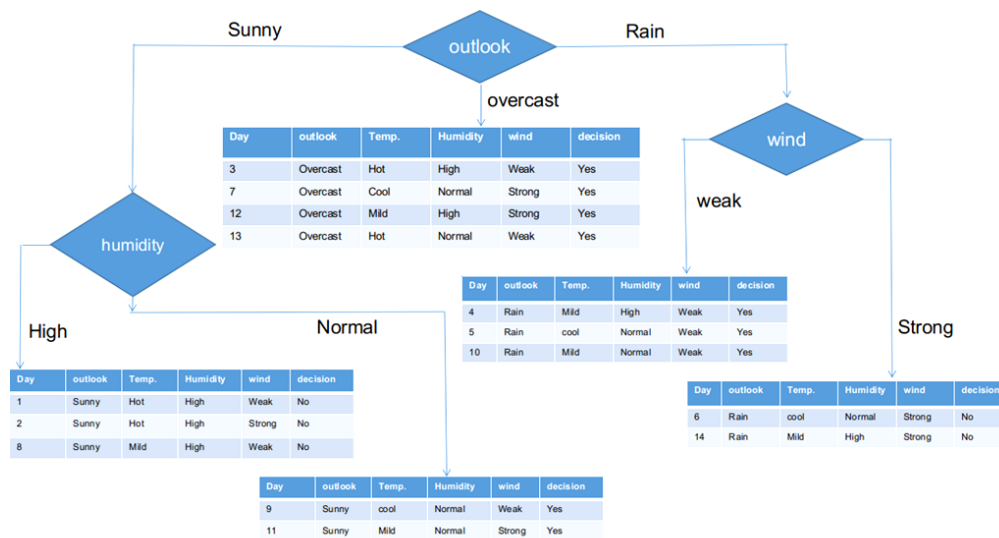
$$= 0 + 0 + 0.408 + 0.408$$

$$= 0.816$$

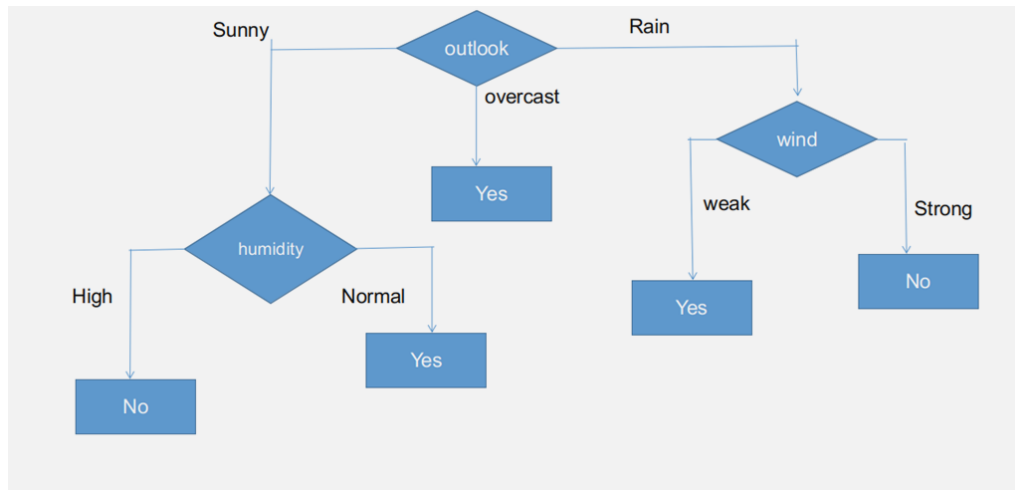
We have found all chi-square values for rain is outlook branch. Let's see them all at a single table.

Feature	Chi-Square Value
Temperature	0.816
Humidity	0.816
Wind	4.449

Thus, the wind feature is the victor for the rain is the outlook branch. Put this column in the connected branch and see the corresponding sub informational dataset.



As seen, all branches have sub informational datasets having a single decision such that yes or no. In this way, we can generate the CHAID tree as illustrated below. The final form of the CHAID tree is given below.



4.3 AUTOMATING MODELS FOR CATEGORICAL AND CONTINUOUS TARGETS:

Predictive modeling is the process by which a model is created to predict an outcome. If the outcome is categorical it is called classification and if the outcome is numerical it is called regression. Classification is a data science task of predicting the value of a categorical variable (target or class) by building a model based on one or more numerical and/or categorical variables (predictors or attributes). Four main groups of classification algorithms are:

1. Frequency Table
 - a. ZeroR
 - b. OneR
 - c. Naive Bayesian
 - d. Decision Tree
2. Covariance Matrix
 - a. Linear Discriminant Analysis
 - b. Logistic Regression
3. Similarity Functions
 - a. K Nearest Neighbors
4. Others
 - a. Artificial Neural Network
 - b. Support Vector Machine

ZeroR

ZeroR is the simplest classification method which relies on the target and ignores all predictors. ZeroR classifier simply predicts the majority category (class).

Although there is no predictability power in ZeroR, it is useful for determining a baseline performance as a benchmark for other classification methods.

Algorithm: Construct a frequency table for the target and select its most frequent value.

Example: "Play Golf = Yes" is the ZeroR model for the following dataset

Predictors				Target
Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Play Golf	
Yes	No
9	5

Predictors Contribution

There is nothing to be said about the predictor's contribution to the model because ZeroR does not use any of them.

Model Evaluation

The following confusion matrix shows that ZeroR only predicts the majority class correctly. As mentioned before, ZeroR is only useful for determining a baseline performance for other classification methods. **Accuracy** = 0.64

Confusion Matrix	Play Golf	
	Yes	No
Yes	9	5
No	0	0

OneR

OneR, short for "One Rule", is a simple, yet accurate, classification algorithm that generates one rule for each predictor in the data, then selects the rule with the smallest total error as its "one rule". To create a rule for a predictor, we construct a frequency table for each predictor against the target. It has been shown that OneR produces rules only slightly less accurate than state-of-the-art classification algorithms

while producing rules that are simple for humans to interpret.

OneR Algorithm

For each predictor,

For each value of that predictor, make a rule as follows;

Count how often each value of target (class) appears

Find the most frequent class

Make the rule assign that class to this value of the predictor

Calculate the total error of the rules of each predictor

Choose the predictor with the smallest total error.

Example:

Finding the best predictor with the smallest total error using OneR algorithm based on related frequency tables.

Naive Bayesian

The Naive Bayesian classifier is based on Bayes' theorem with the independence assumptions between predictors. A Naive Bayesian model is easy to build, with no complicated iterative parameter estimation which makes it particularly useful for very large datasets. Despite its simplicity, the Naive Bayesian classifier often does surprisingly well and is widely used because it often outperforms more sophisticated classification methods.

Algorithm

Bayes theorem provides a way of calculating the posterior probability, $P(c|x)$, from $P(c)$, $P(x)$, and $P(x|c)$. Naive Bayes classifier assumes that the effect of the value of a predictor (x) on a given class (c) is independent of the values of other predictors. This assumption is called class conditional independence.

$$P(c | x) = \frac{P(x | c) P(c)}{P(x)}$$

Likelihood
Class Prior Probability

Posterior Probability
Predictor Prior Probability

$$P(c | X) = P(x_1 | c) \times P(x_2 | c) \times \dots \times P(x_n | c) \times P(c)$$

- $P(c|x)$ is the posterior probability of class (target) given predictor (attribute).
- $P(c)$ is the prior probability of class.
- $P(x|c)$ is the likelihood which is the probability of predictor given class.
- $P(x)$ is the prior probability of predictor.

In ZeroR model there is no predictor, in OneR model we try to find the single best predictor, naive Bayesian includes all predictors using Bayes' rule and the independence assumptions between predictors.

Decision Tree - Classification

Decision tree builds classification or regression models in the form of a tree structure. It breaks down a dataset into smaller and smaller subsets while at the same time an associated decision tree is incrementally developed. The final result is a tree with **decision nodes** and **leaf nodes**. A decision node (e.g., Outlook) has two or more branches (e.g., Sunny, Overcast and Rainy). Leaf node (e.g., Play) represents a classification or decision. The topmost decision node in a tree which corresponds to the best predictor called **root node**. Decision trees can handle both categorical and numerical data.

K Nearest Neighbors – Classification

K nearest neighbors is a simple algorithm that stores all available cases and classifies new cases based on a similarity measure (e.g., distance functions). KNN has been used in statistical estimation and pattern recognition already in the beginning of 1970's as a non-parametric technique.

Algorithm

A case is classified by a majority vote of its neighbors, with the case being

assigned to the class most common amongst its K nearest neighbors measured by a distance function. If $K = 1$, then the case is simply assigned to the class of its nearest neighbor. It should also be noted that all three distance measures are only valid for continuous variables. In the instance of categorical variables the Hamming distance must be used. It also brings up the issue of standardization of the numerical variables between 0 and 1 when there is a mixture of numerical and categorical variables in the dataset. Choosing the optimal value for K is best done by first inspecting the data. In general, a large K value is more precise as it reduces the overall noise but there is no guarantee. Cross-validation is another way to retrospectively determine a good K value by using an independent dataset to validate the K value. Historically, the optimal K for most datasets has been between 3 and 10. That produces much better results than 1NN.

Regression

Regression is a data science task of predicting the value of target (numerical variable) by building a model based on one or more predictors (numerical and categorical variables).

1. Frequency Table
 - Decision Tree
2. Covariance Matrix
 - Multiple Linear Regression
3. Similarity Function
 - K Nearest Neighbors
4. Others
 - Artificial Neural Network
 - Support Vector Machine

4.4 COMPARING AND COMBINING MODELS:

An ensemble method is a machine learning technique that combines several base models in order to produce one optimal predictive model. When you want to purchase a new car, will you walk up to the first car shop and purchase one based on the advice of the dealer? It's highly unlikely. You would likely browse a few web portals where people have posted their reviews and compare different car models, checking for their features and prices. You will also probably ask your friends and

colleagues for their opinion. In short, you wouldn't directly reach a conclusion, but will instead make a decision considering the opinions of other people as well. Ensemble models in machine learning operate on a similar idea. They combine the decisions from multiple models to improve the overall performance.

Ensemble Techniques

1. Max Voting
2. Averaging
3. Weighted Averaging

Max Voting

The max voting method is generally used for classification problems. In this technique, multiple models are used to make predictions for each data point. The predictions by each model are considered as a 'vote'. The predictions which we get from the majority of the models are used as the final prediction. For example, when you asked 5 of your colleagues to rate your movie (out of 5); we'll assume three of them rated it as 4 while two of them gave it a 5. Since the majority gave a rating of 4, the final rating will be taken as 4. We can consider this as taking the mode of all the predictions. The result of max voting would be something like this:

Colleague 1	Colleague 2	Colleague 3	Colleague 4	Colleague 5	Final rating
5	4	5	4	4	4

Averaging

Similar to the max voting technique, multiple predictions are made for each data point in averaging. In this method, we take an average of predictions from all the models and use it to make the final prediction. Averaging can be used for making predictions in regression problems or while calculating probabilities for classification problems. For example, in the below case, the averaging method would take the average of all the values. i.e. $(5+4+5+4+4)/5 = 4.4$

Colleague 1	Colleague 2	Colleague 3	Colleague 4	Colleague 5	Final rating
5	4	5	4	4	4.4

Weighted Average

This is an extension of the averaging method. All models are assigned different

weights defining the importance of each model for prediction. For instance, if two of your colleagues are critics, while others have no prior experience in this field, then the answers by these two friends are given more importance as compared to the other people.

The result is calculated as $[(5 \times 0.23) + (4 \times 0.23) + (5 \times 0.18) + (4 \times 0.18) + (4 \times 0.18)] = 4.41$.

Colleague 1		Colleague 2	Colleague 3	Colleague 4	Colleague 5	Final rating
Weight	0.23	0.23	0.18	0.18	0.18	
Rating	5	4	5	4	4	4.41

Need for Ensembling

There are two main reasons to use an ensemble over a single model, and they are related; they are:

1. Performance: An ensemble can make better predictions and achieve better performance than any single contributing model.

2. Robustness: An ensemble reduces the spread or dispersion of the predictions and model performance.

Ensembles are used to achieve better predictive performance on a predictive modeling problem than a single predictive model.

Applications of Ensemble learning:

Ensemble based systems can be used in problem domains other than improving the generalization performance of a classifier.

1. Incremental learning

Incremental learning refers to the ability of an algorithm to learn from new data that may become available after a classifier (or a model) has already been generated from a previously available dataset. An incremental learning algorithm must learn the new information, and retain previously acquired knowledge, without having access to previously seen data. Ensemble based systems can be used for such problems by training an additional classifier (or an additional ensemble of classifiers) on each dataset that becomes available.

2. Error correcting output codes

Error correcting output codes (ECOC) are used in machine learning for converting binary classifiers, such as support vector machines, to multi-class classifiers by decomposing a multi-class problem into several two-class problems. The idea is to use a different class encoding for each member of the ensemble.

3. Feature selection

As mentioned earlier in this article, one way to improve diversity in the ensemble is to train individual classifiers different subsets of the available features. Selecting the feature subsets at random is known as the random subspace method. Here are some of the **real-life applications** of Ensemble Learning.

Disease detection : Classification and localization of diseases for simplistic and fast prognosis have been aided by Ensemble learning, like in cardiovascular disease detection from X-Ray and CT scans.

Remote Sensing: Monitoring of physical characteristics of a target area without coming in physical contact, called Remote Sensing, is a difficult task since the data acquired by different sensors have varying resolutions leading to incoherence in data distribution. Tasks like Landslide Detection and Scene Classification have also been accomplished with the help of Ensemble Learning.

Fraud Detection: Detection of digital fraud is an important and challenging task since very minute precision is required to automate the process. Ensemble Learning has proved its efficacy in detecting Credit Card Fraud and Impression Fraud.

Speech emotion recognition: Ensemble Learning is also applied in speech emotion recognition, especially in the case of multi-lingual environments. The technique allows for the combining of all classifiers' effect instead of choosing one classifier and compromising certain language corpus's accuracy.

Types of ensemble methods

There are many ensemble techniques available but we will discuss about the below two mostwidely used methods:

1. Bagging
2. Boosting
3. Stacking

4. Blending

Similarities between bagging and boosting methods

Both the methods can be used for classification (discrete output) and regression (continuous output) problems. Both methods are better in performance compared to single models as they aggregate the output of all the weak learners via voting thereby leading to more accuracy in the predictions.

Bagging:

Also known as Bootstrap aggregating because the training set is divided into multiple sub samples and fed to each weak learner (for e.g.: each decision tree). And each sub sample contains a few set of features along with a few observations selected randomly.

Example: We all attend interviews right and the interview process is the simplest scenario I could come across which is analogous to Bagging. Suppose you went to attend an interview and as soon as you enter the room, you see five stern looking people sitting there to lash out all their questions at you. What do you do? Feel nervous and storm out of the room!! No right? You went there because you need the job and you know you are bold enough, intelligent enough to face whatever comes your way and you greet them anyway and sit down in front of them. So, let's face this. Now consider each of the five interviewers are going to interview the candidate on five different aspects say aptitude, technical -basic, technical- advanced, communication and management skills. Also, the company might already have prepared a set of questions based on the different aspects to test the candidate's skills. And let's name this as 'question set'. So, the interview starts now. And our target variable here would be if the candidate gets selected or not.

Interviewer 1 — The interviewer 1 starts testing your aptitude skills taking a sample of aptitude based questions from the 'question set' and sets some probabilities whether you should be selected or not.

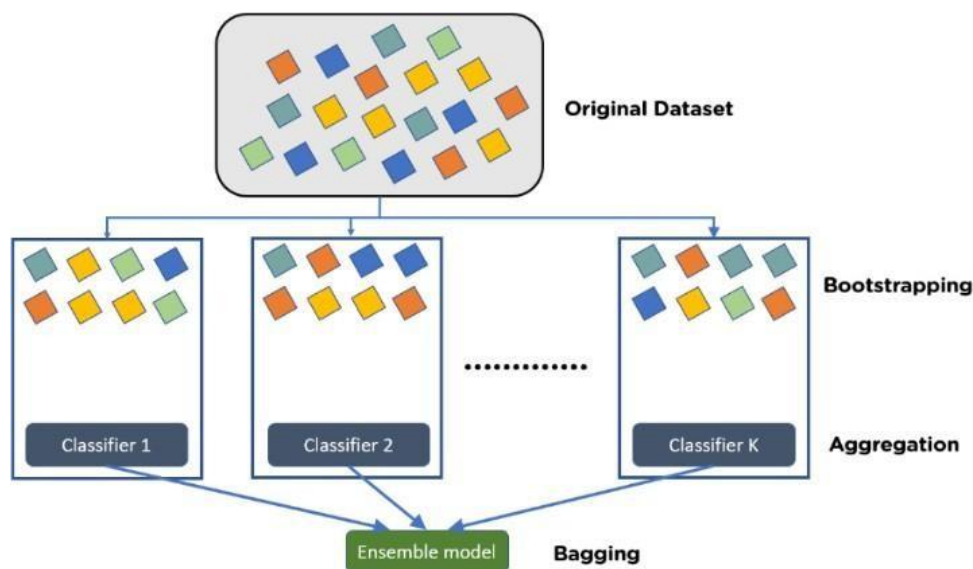
Interviewer 2 — The interviewer 2 also starts testing your basic technical skills, again taking a sample of preliminary technical questions from the 'question set' and sets some probabilities whether you should be selected or not.

Interviewer 3 — Similarly, the interviewer 3 starts testing your advanced technical skills taking another sample of slightly difficult technical questions from

the 'question set' and again sets some probabilities whether you should be selected or not.

Interviewer 4 & Interviewer 5: Interviewer 4 and interviewer 5 repeats the same process as above, testing the candidate's communication and management skills taking a sample of related questions from the 'question set' and accordingly sets some probabilities whether to select or not.

Result: Finally the candidate is done with the interview and he comes out of the room leaving the interviewers to analyze their respective scores about his skills based on the sub samples of questions from the 'question set' that he was scored upon . Ultimately they aggregate all of their individual opinions and come out with the final decision whether to select the candidate or not. This is exactly similar to ensemble methods.



Bagging, also known as Bootstrap aggregating, is an ensemble learning technique that helps to improve the performance and accuracy of machine learning algorithms. It is used to deal with bias-variance trade-offs and reduces the variance of a prediction model. Bagging avoids overfitting of data and is used for both regression and classification models, specifically for decision tree algorithms.

Steps to Perform Bagging

- i. Consider there are n observations and m features in the training set. You need to select a random sample from the training dataset without replacement.
- ii. A subset of m features is chosen randomly to create a model using sample observations
- iii. The feature offering the best split out of the lot is used to split the nodes
- iv. The tree is grown, so you have the best root nodes
- v. The above steps are repeated n times. It aggregates the output of individual decision trees to give the best prediction.

Advantages of Bagging

- i. Bagging minimizes the overfitting of data
- ii. It improves the model's accuracy
- iii. It deals with higher dimensional data efficiently

Boosting

Boosting is a sequential process, where each subsequent model attempts to correct the errors of the previous model. The succeeding models are dependent on the previous model. Let's understand this definition in detail by solving a problem of spam email identification: How would you classify an email as SPAM or not? Like everyone else, our initial approach would be to identify 'spam' and 'not spam' emails using following criteria. If:

1. Email has only one image file (promotional image), It's a SPAM
2. Email has only link(s), It's a SPAM
3. Email body consists of sentence like "You won a prize money of \$ xxxxxx", It's a SPAM
4. Email from our official domain "abc.com", Not a SPAM
5. Email from known source, Not a SPAM

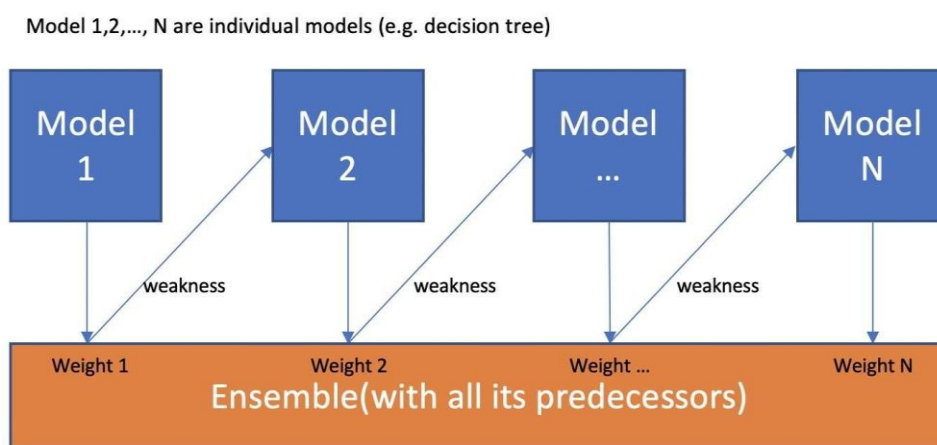
Above, we've defined multiple rules to classify an email into 'spam' or 'not spam'. But, do you think these rules individually are strong enough to successfully classify an email? No. Individually, these rules are not powerful enough to classify an email into 'spam' or 'not spam'. Therefore, these rules are called as **weak learner**. To convert weak learner to strong learner, we'll combine the prediction of each weak

learner using methods like:

- Using average/ weighted average
- Considering prediction has higher vote

For example: Above, we have defined 5 weak learners. Out of these 5, 3 are voted as 'SPAM' and 2 are voted as 'Not a SPAM'. In this case, by default, we'll consider an email as SPAM because we have higher (3) vote for 'SPAM'.

How Boosting Algorithms works?



Now we know that, boosting combines weak learner a.k.a. base learner to form a strong rule. An immediate question which should pop in your mind is, '*How boosting identify weak rules?*' To find weak rule, we apply base learning (ML) algorithms with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After much iteration, the boosting algorithm combines these weak rules into a single strong prediction rule. Here's another question which might haunt you, '*How do we choose different distribution for each round?*' For choosing the right distribution, here are the following steps:

Step 1: The base learner takes all the distributions and assigns equal weight or attention to each observation.

Step 2: If there is any prediction error caused by first base learning algorithm, then we pay higher attention to observations having prediction error. Then, we apply the next base learning algorithm.

Step 3: Iterate Step 2 till the limit of base learning algorithm is reached or higher accuracy is achieved.

Finally, it combines the outputs from weak learner and creates a strong learner which eventually improves the prediction power of the model. Boosting pays higher focus on examples which are mis-classified or have higher errors by preceding weak rules.

Differences between bagging and boosting:

Bagging	Boosting
Individual trees/models are independent of each other.	Individual trees are not independent of each other.
There is no concept of learning from each other in bagging. Each individual model/tree will be fed with a sample of features/columns from the whole training set along with a sample of observations/rows for those features.	In boosting, each of the trees will learn from the mistakes of the previous tree and try to minimize the residual error as it keeps moving forward sequentially. For e.g.: I have a boosting algorithm with 3 sequential models M1, M2 and M3. M2 tries to reduce the residual error generated by M1 and M3 will try to bring down the residual error generated by the M2 close to zero thereby giving the best accuracy.
Learning rate is not used as a hyper parameter in Bagging methods as the trees are independent of each other.	Learning rate is used as a hyper parameter in Boosting methods as each of the trees learns from the previous iterations.
Examples: Random Forest, Extra Tree algorithms.	Examples: Gradient Boosting, ADABOOST, XGBoost.
Helps reduce variance.	Helps reduce both bias and variance.

Types of Boosting Algorithms

Underlying engine used for boosting algorithms can be anything. It can be decision stamp, margin-maximizing classification algorithm etc. There are many boosting algorithms which use other types of engine such as:

1. AdaBoost (**Adaptive Boosting**)
2. Gradient Tree Boosting
3. XGBoost

Stacking

Stacking, also known as Stacked Generalization is an ensemble technique that combines multiple classifications or regression models via a meta-classifier or a meta-regressor. The base-level models are trained on a complete training set, and then the meta-model is trained on the features that are outputs of the base-level model. The base-level often consists of different learning algorithms and therefore stacking ensembles are often heterogeneous.

Blending

Blending is also an ensemble technique that can help us to improve performance and increase accuracy. It follows the same approach as stacking but uses only a holdout (validation) set from the train set to make predictions. In other words, unlike stacking, the predictions are made on the holdout set only. The holdout set and the predictions are used to build a model which is run on the test set. Here is a detailed explanation of the blending process:

1. The train set is split into two parts, viz-training and validation sets.
2. Model(s) are fit on the training set.
3. The predictions are made on the validation set and the test set.
4. The validation set and its predictions are used as features to build a new model.
5. This model is used to make final predictions on the test and meta-features.

The difference between stacking and blending is that Stacking uses out-of-fold predictions for the train set of the next layer (i.e meta-model), and Blending uses a validation set (let's say, 10-15% of the training set) to train the next layer.

4.5 EVALUATION CHARTS FOR MODEL COMPARISON:

Model Evaluation is an integral part of the model development process. It helps to find the best model that represents our data and how well the chosen model will work in the future. Evaluating model performance with the data used for training is not acceptable in data science because it can easily generate overoptimistic and overfitted models. There are two methods of evaluating models in data science, Hold-Out and Cross-Validation. To avoid overfitting, both methods use a test set (not seen by the model) to evaluate model performance.

Hold-Out

In this method, the mostly large dataset is *randomly* divided to three subsets:

1. **Training set** is a subset of the dataset used to build predictive models.
2. **Validation set** is a subset of the dataset used to assess the performance of model built in the training phase. It provides a test platform for fine tuning model's parameters and selecting the best-performing model. Not all modeling algorithms need a validation set.
3. **Test set** or unseen examples are a subset of the dataset to assess the likely future performance of a model. If a model fit to the training set much better than it fits the test set, overfitting is probably the cause.

Cross-Validation

When only a limited amount of data is available, to achieve an unbiased estimate of the model performance we use k -fold cross-validation. In k -fold cross-validation, we divide the data into k subsets of equal size. We build models k times, each time leaving out one of the subsets from training and use it as the test set. If k equals the sample size, this is called "leave-one-out".

Model evaluation can be divided to two sections:

1. Classification Evaluation
2. Regression Evaluation

MODEL EVALUATION – CLASSIFICATION:

1. Confusion matrix
2. Accuracy
3. Precision
4. Recall
5. Specificity
6. F1 score
7. **ROC (Receiver Operating Characteristics) curve**

Confusion Matrix

		Predicted class		Total
		<i>yes</i>	<i>no</i>	
Actual class	<i>yes</i>	<i>TP</i>	<i>FN</i>	<i>P</i>
	<i>no</i>	<i>FP</i>	<i>TN</i>	<i>N</i>
Total		<i>P'</i>	<i>N'</i>	<i>P + N</i>

A confusion matrix shows the number of correct and incorrect predictions made by the classification model compared to the actual outcomes (target value) in the data. The matrix is $N \times N$, where N is the number of target values (classes). Performance of such models is commonly evaluated using the data in the matrix. The following table displays a 2x2 confusion matrix for two classes (Positive and Negative). There are four additional terms we need to know that are the “building blocks” used in computing many evaluation measures.

True Positives (TP): These refer to the positive tuples that were correctly labeled by the classifier.

True Negatives (TN): These are the negative tuples that were correctly labeled by the classifier.

False Positives (FP): These are the negative tuples that were incorrectly labeled as positive.

False Negatives (FN): These are the positive tuples that were mislabeled as negative.

Confusion Matrix		Target			
		Positive	Negative		
Model	Positive	a (TP)	b (FN)	Positive Predictive Value	$a/(a+b)$
	Negative	c (FP)	d (TN)	Negative Predictive Value	$d/(c+d)$
		Sensitivity	Specificity	Accuracy = $(a+d)/(a+b+c+d)$	
		$a/(a+c)$	$d/(b+d)$		

Accuracy : the proportion of the total number of predictions that were correct.

Positive Predictive Value or Precision : the proportion of positive cases that were correctly identified.

Negative Predictive Value : the proportion of negative cases that were correctly identified.

Sensitivity or Recall : the proportion of actual positive cases which are correctly identified.

Specificity : the proportion of actual negative cases which are correctly identified.

Example:

Confusion Matrix		Target			
		Positive	Negative		
Model	Positive	70	20	Positive Predictive Value	0.78
	Negative	30	80	Negative Predictive Value	0.73
		Sensitivity	Specificity	Accuracy = 0.75	
		0.70	0.80		

<i>Measure</i>	<i>Formula</i>
accuracy, recognition rate	$\frac{TP + TN}{P + N}$
error rate, misclassification rate	$\frac{FP + FN}{P + N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP + FP}$
F , F_1 , F -score, harmonic mean of precision and recall	$\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$
F_β , where β is a non-negative real number	$\frac{(1 + \beta^2) \times \text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}}$

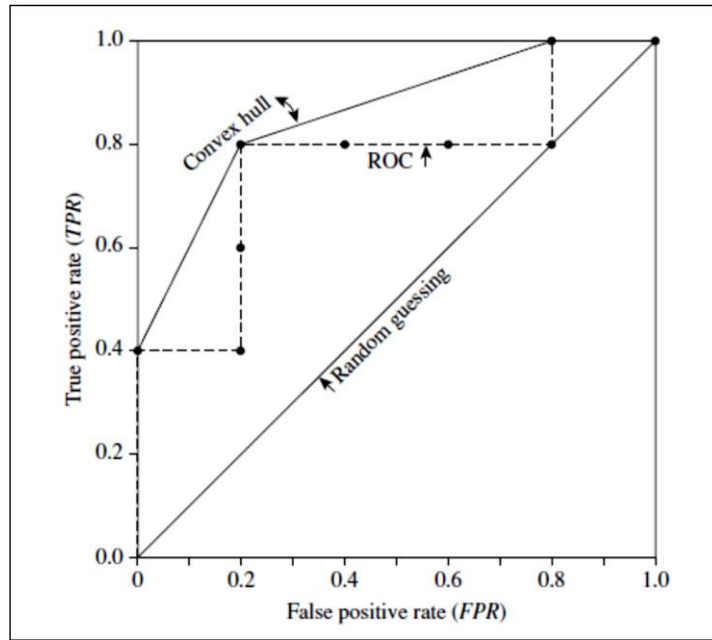
Plotting an ROC curve:

<i>Tuple #</i>	<i>Class</i>	<i>Prob.</i>	<i>TP</i>	<i>FP</i>	<i>TN</i>	<i>FN</i>	<i>TPR</i>	<i>FPR</i>
1	<i>P</i>	0.90	1	0	5	4	0.2	0
2	<i>P</i>	0.80	2	0	5	3	0.4	0
3	<i>N</i>	0.70	2	1	4	3	0.4	0.2
4	<i>P</i>	0.60	3	1	4	2	0.6	0.2
5	<i>P</i>	0.55	4	1	4	1	0.8	0.2
6	<i>N</i>	0.54	4	2	3	1	0.8	0.4
7	<i>N</i>	0.53	4	3	2	1	0.8	0.6
8	<i>N</i>	0.51	4	4	1	1	0.8	0.8
9	<i>P</i>	0.50	5	4	0	1	1.0	0.8
10	<i>N</i>	0.40	5	5	0	0	1.0	1.0

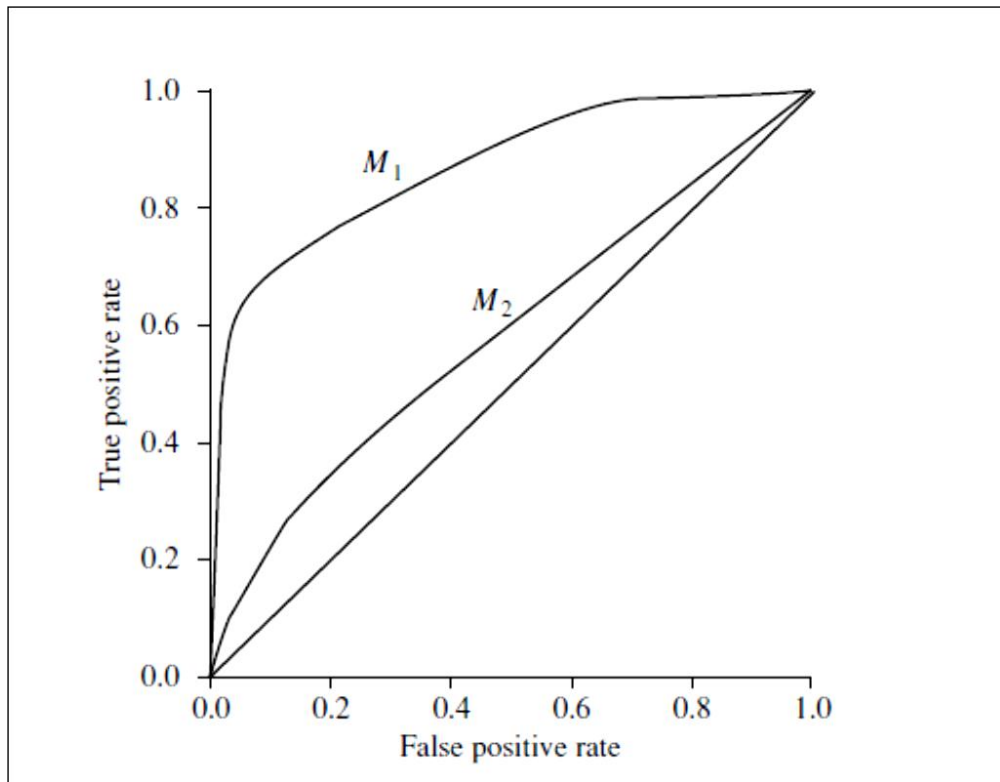
Table shows the probability value (column 3) returned by a probabilistic classifier for each of the 10 tuples in a test set, sorted by decreasing probability order. Column 1 is merely a tuple identification number, which aids in our explanation. Column 2 is the actual class label of the tuple. There are five positive tuples and five negative tuples, thus $P = 5$ and $N = 5$. As we examine the known class label of each tuple, we can determine the values of the remaining columns, TP , FP , TN , FN , TPR , and FPR . We start with tuple 1, which has the highest probability score, and take that score as our threshold, that is, $t = 0.9$. Thus, the classifier considers tuple 1 to be positive, and all the other tuples are considered negative. Since the actual class label of tuple 1 is positive, we have a true positive, hence $TP = 1$ and $FP = 0$. Among the remaining nine tuples, which are all classified as negative, five actually are negative (thus, $TN = 5$). The remaining four are all actually positive, thus, $FN = 4$. We can therefore compute $TPR = TP/P = 1/5 = 0.2$, while $FPR = 0$. Thus, we have the point (0.2, 0) for the ROC curve.

Next, threshold t is set to 0.8, the probability value for tuple 2, so this tuple is now also considered positive, while tuples 3 through 10 are considered negative. The actual

class label of tuple 2 is positive, thus now $TP = 2$. The rest of the row can easily be computed, resulting in the point (0.4, 0). Next, we examine the class label of tuple 3 and let t be 0.7, the probability value returned by the classifier for that tuple. Thus, tuple 3 is considered positive, yet its actual label is negative, and so it is a false positive. Thus, TP stays the same and FP increments so that $FP = 1$. The rest of the values in the row can also be easily computed, yielding the point (0.4, 0.2). The resulting ROC graph, from examining each tuple, is the jagged line shown in Figure.



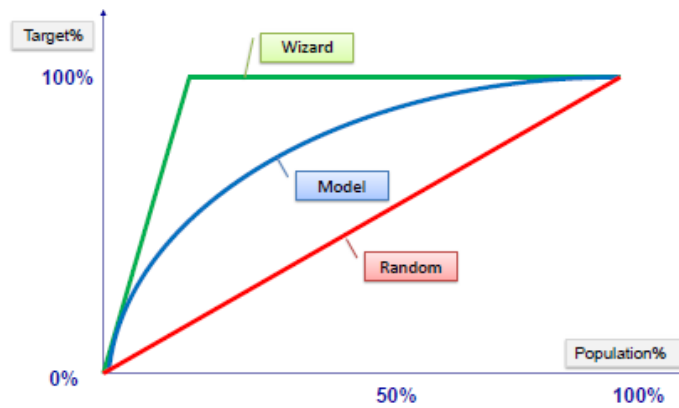
There are many methods to obtain a curve out of these points, the most common of which is to use a convex hull. The plot also shows a diagonal line where for every true positive of such a model, we are just as likely to encounter a false positive. For comparison, this line represents random guessing. Figure shows the ROC curves of two classification models. The diagonal line representing random guessing is also shown. Thus, the closer the ROC curve of a model is to the diagonal line, the less accurate the model. If the model is really good, initially we are more likely to encounter true positives as we move down the ranked list. Thus, the curve moves steeply up from zero. Later, as we start to encounter fewer and fewer true positives, and more and more false positives, the curve eases off and becomes more horizontal.



To assess the accuracy of a model, we can measure the area under the curve. Several software packages are able to perform such calculation. The closer the area is to 0.5, the less accurate the corresponding model is. A model with perfect accuracy will have an area of 1.0.

Gain and Lift Charts

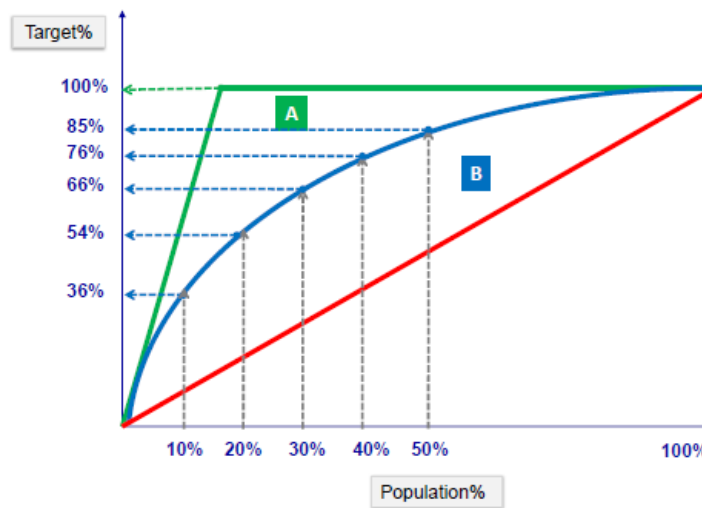
Gain or lift is a measure of the effectiveness of a classification model calculated as the ratio between the results obtained with and without the model. Gain and lift charts are visual aids for evaluating performance of classification models. However, in contrast to the confusion matrix that evaluates models on the whole population gain or lift chart evaluates model performance in a portion of the population.



Example:

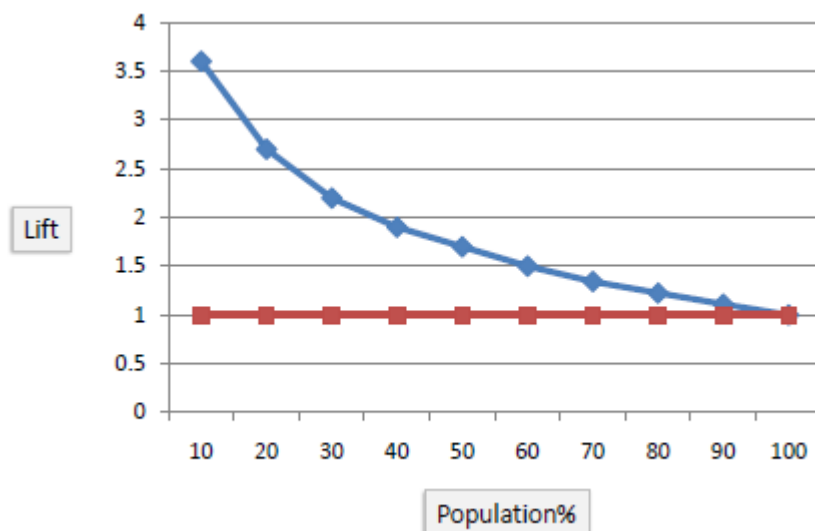
Score Table		Sorted by Score		Gain Table		Lift Table	
Target	Score	Target	Score	Count%	Target%	Count%	Lift
0	235	1	880	10	36	10	3.6
1	724	1	724	20	54	20	2.7
1	556	1	676	30	66	30	2.2
0	345	1	556	40	76	40	1.9
0	480	0	480	50	85	50	1.7
1	676	0	368	60	90	60	1.5
0	195	0	345	70	94	70	1.3
1	880	0	235	80	98	80	1.2
0	368	0	195	90	100	90	1.1
...	...	...	...	100	100	100	1

Gain Chart



Lift Chart

The lift chart shows how much more likely we are to receive positive responses than if we contact a random sample of customers. For example, by contacting only 10% of customers based on the predictive model we will reach 3 times as many respondents, as if we use no model.



K-S Chart

K-S or Kolmogorov-Smirnov chart measures performance of classification models. More accurately, K-S is a measure of the degree of separation between the positive and negative distributions. The K-S is 100 if the scores partition the population into two separate groups in which one group contains all the positives and the other all the negatives. On the other hand, If the model cannot differentiate between positives and negatives, then it is as if the model selects cases randomly from the population. The K-S would be 0. In most classification models the K-S will fall between 0 and 100, and that the higher the value the better the model is at separating the positive from negative cases.

Example:

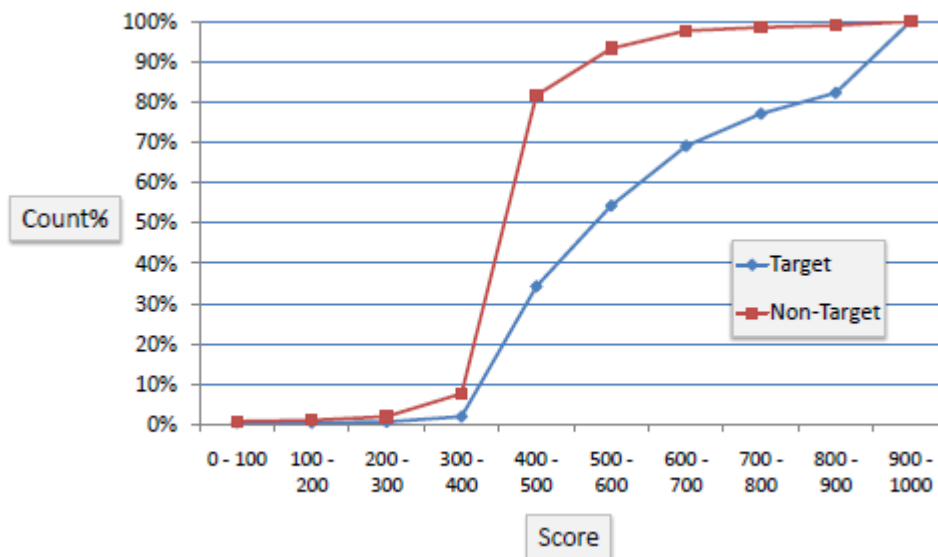
The following example shows the results from a classification model. The model

assigns a score between 0-1000 to each positive (Target) and negative (Non-Target) outcome.

Score Range		Count		Cumulative Count		K-S
Lower	Upper	Target	Non-Target	Target	Non-Target	
0	100	3	62	0.5%	0.8%	0.3%
100	200	0	23	0.5%	1.1%	0.6%
200	300	1	66	0.7%	2.0%	1.3%
300	400	7	434	2.0%	7.7%	5.7%
400	500	181	5627	34.3%	81.7%	47.4%
500	600	112	886	54.3%	93.3%	39.0%
600	700	83	332	69.1%	97.7%	28.6%
700	800	45	63	77.1%	98.5%	21.4%
800	900	29	37	82.3%	99.0%	16.7%
900	1000	99	77	100.0%	100.0%	0.0%

K-S

K(0.95) = 6.0%
 K(0.99) = 7.1%



Area Under the Curve (AUC)

Area under ROC curve is often used as a measure of quality of the classification models. A random classifier has an area under the curve of 0.5, while AUC for a perfect

Q1. Scenario: Online Learning Platform — Course Completion Analysis

An online education company wants to analyze **student course completion behavior** to improve retention strategies.

Dataset

Student	Study Mode	Weekly Study Hours	Device Used	Course Completed
S1	Part-time	Low	Mobile	No
S2	Full-time	High	Laptop	Yes
S3	Part-time	Low	Mobile	No
S4	Full-time	Medium	Laptop	Yes
S5	Part-time	Medium	Tablet	No
S6	Full-time	High	Laptop	Yes
S7	Part-time	High	Laptop	Yes
S8	Full-time	Medium	Mobile	Yes

Question (CHAID Analysis)

Using **CHAID (Chi-square Automatic Interaction Detection)** concept:

1. Identify all **categorical predictor variables**.
2. Perform **categorical splits** based on differences in *Course Completed*.
3. Determine **which attribute should become the root node** using category significance differences.

Q2. Scenario: Bank Loan Risk Classification

A bank wants to classify customers into **Low Risk** or **High Risk** categories based on their financial indicators before approving loans.

Dataset

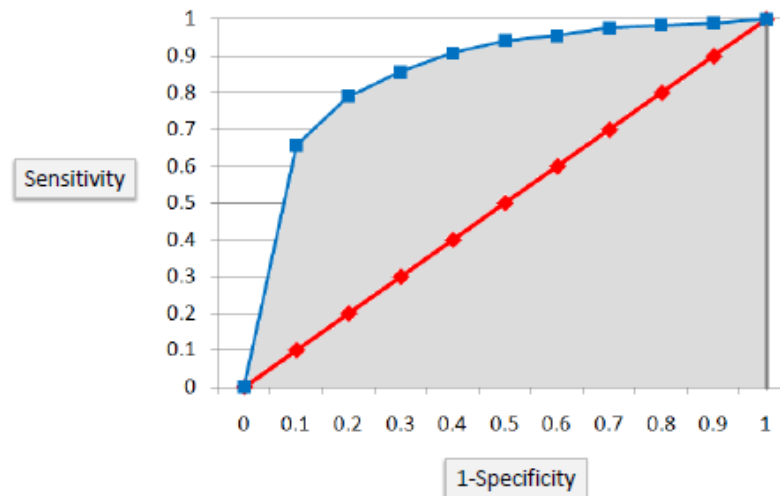
Customer	Income Score	Credit Score	Risk Category
C1	82	78	Low Risk
C2	45	50	High Risk
C3	76	72	Low Risk

C4	52	48	High Risk
C5	80	75	Low Risk
C6	48	55	High Risk

Question

Explain how **Linear Discriminant Analysis (LDA)** separates the two groups (**Low Risk** and **High Risk**).

classifier is equal to 1. In practice, most of the classification models have an AUC between 0.5 and 1.



An area under the ROC curve of 0.8, for example, means that a randomly selected case from the group with the target equals 1 has a score larger than that for a randomly chosen case from the group with the target equals 0 in 80% of the time. When a classifier cannot distinguish between the two groups, the area will be equal to 0.5 (the ROC curve will coincide with the diagonal). When there is a perfect separation of the two groups, i.e., no overlapping of the distributions, the area under the ROC curve reaches to 1 (the ROC curve will reach the upper left corner of the plot).

MODEL EVALUATION – REGRESSION:

Root Mean Squared Error: RMSE is a popular formula to measure the error rate of a regression model. However, it can only be compared between models whose errors are measured in the same units.

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (p_i - a_i)^2}{n}}$$

a = actual target
 p = predicted target

Relative Squared Error: Unlike RMSE, the relative squared error (RSE) can be compared between models whose errors are measured in the different units.

$$RSE = \frac{\sum_{i=1}^n (p_i - a_i)^2}{\sum_{i=1}^n (\bar{a} - a_i)^2}$$

Mean Absolute Error: The mean absolute error (MAE) has the same unit as the original data, and it can only be compared between models whose errors are measured in the same units. It is usually similar in magnitude to RMSE, but slightly smaller.

$$MAE = \frac{\sum_{i=1}^n |p_i - a_i|}{n}$$

Relative Absolute Error: Like RSE , the relative absolute error (RAE) can be compared between models whose errors are measured in the different units.

$$RAE = \frac{\sum_{i=1}^n |p_i - a_i|}{\sum_{i=1}^n |\bar{a} - a_i|}$$

Coefficient of Determination: The coefficient of determination (R²) summarizes the explanatory power of the regression model and is computed from the sums-of-squares terms.

$$\text{Coefficient of Determination} \rightarrow R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

$$\text{Sum of Squares Total} \rightarrow SST = \sum (y - \bar{y})^2$$

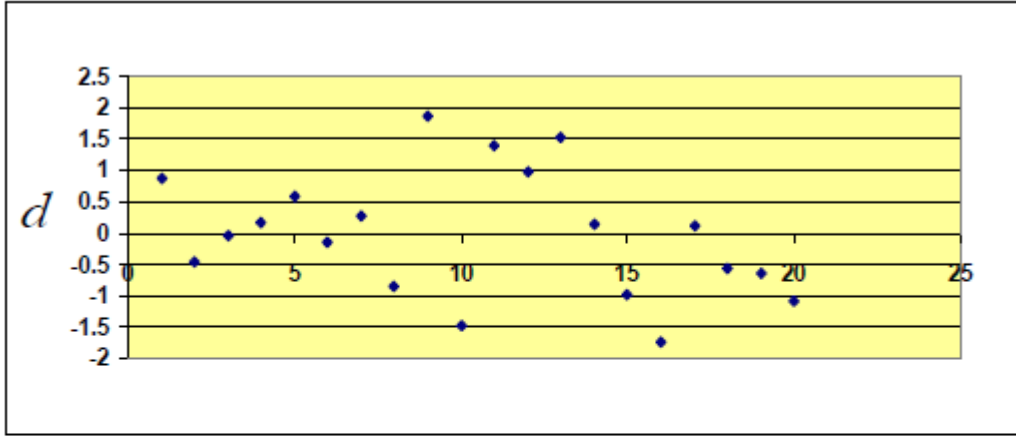
$$\text{Sum of Squares Regression} \rightarrow SSR = \sum (y' - \bar{y}')^2$$

$$\text{Sum of Squares Error} \rightarrow SSE = \sum (y - y')^2$$

R² describes the proportion of variance of the dependent variable explained by the regression model. If the regression model is “perfect”, SSE is zero, and R² is 1. If the regression model is a total failure, SSE is equal to SST, no variance is explained by regression, and R² is zero.

Standardized Residuals (Errors) Plot:

The standardized residual plot is a useful visualization tool in order to show the residual dispersion patterns on a standardized scale. There are no substantial differences between the pattern for a standardized residual plot and the pattern in the regular residual plot. The only difference is the standardized scale on the y-axis which allows us to easily detect potential outliers.



$$d_i = \frac{e_i}{S_e}$$

$e_i = y_i - y'_i$
 $S_e = \sqrt{\frac{SSE}{n-k-1}}$

n = number of observations
k = number of predictors

4.6 META LEVEL MODELING:

The field of meta-learning, or learning-to-learn, has seen a dramatic increase in interest in recent years. Unlike conventional approaches to AI, where tasks are solved from scratch using a fixed learning algorithm, meta-learning aims to improve the algorithm, considering the experience of multiple learning episodes. This paradigm provides an opportunity to address many of the conventional challenges of deep learning, including data and computational bottlenecks and generalization. This survey describes the current landscape of meta-learning. We first discuss definitions of meta-learning and situate them concerning related fields such as transfer learning and hyperparameter optimization. We then propose a new taxonomy that provides a more comprehensive partitioning of the space of today's meta-learning methods. We explore meta-learning's promising applications and achievements, such as multiple trials and reinforcement learning. The performance of a learning model depends on its training dataset, algorithm, and

parameters. Many experiments are needed to find the best performing algorithm and algorithm parameters. Meta-learning approaches help them find and optimize the number of experiments. The result is better predictions in less time.

Meta-learning can be used for various machine learning models (e.g., few-shot, Reinforcement Learning, natural language processing, etc.). Meta-learning algorithms make predictions by inputting the outputs and metadata of machine learning algorithms. Meta-learning algorithms can learn to use the best predictions from machine learning algorithms to make better predictions.

What is Meta-learning?

Meta-learning, described as “learning to learn”, is a subset of machine learning in the field of computer science. It is used to improve the results and performance of the learning algorithm by changing some aspects of the learning algorithm based on the results of the experiment. Meta-learning helps researchers understand which algorithms generate the best/better predictions from datasets. Meta-learning algorithms use learning algorithm metadata as input. They then make predictions and provide information about the performance of these learning algorithms as output. An image’s metadata in a learning model can include, for example, its size, resolution, style, creation date, and owner. Systematic experiment design in meta-learning is the most important challenge. Machine learning algorithms have some problems, such as

- The need for large datasets for training
- High operating costs due to many trials/experiments during the training phase
- Experiments/trials take a long time to find the best model that performs best for a given data set.

Meta-learning can help machine learning algorithms deal with these challenges by optimizing and finding learning algorithms that perform better.

What is Interest in Meta-learning?

Interest in meta-learning has been growing over the last five years, especially after 2017, it accelerated. As the use of deep learning and advanced machine learning algorithms has increased, the difficulty of training these algorithms has increased interest in meta-learning studies.

Working of Meta-learning

In general, a meta-learning algorithm is trained using the outputs (i.e., model predictions) and metadata of machine learning algorithms. After training is done, its skills are sent for testing and used to make end/final predictions. Meta-learning includes tasks such as

- Observing the performance of different machine learning models on learning tasks
- Learning from metadata
- The faster learning process for new tasks

For example, we may want to train a machine learning model to label discrete breeds of dogs.

- i. First, we need an annotated dataset.
- ii. Different ML models are built on the training set. They could only focus on certain parts of the dataset.
- iii. A meta-training process is used to improve the performance of these models
- iv. Finally, the meta-training model can be used to build a new model from several examples based on its experience with the previous training process

Advantages of Meta-learning

Meta-learning algorithms are widely used to enhance machine learning solutions. The benefits of meta-learning are

Higher model prediction accuracy:

- Optimization of learning algorithms: For example, optimization of hyperparameters to achieve the best results. Thus, this optimization task, which is normally performed by a human, is performed using a meta-learning algorithm.
- Helps to learn algorithms better adapt to changing conditions
- Identifying clues for designing better learning algorithms

The faster and cheaper training process

- Supporting learning from fewer examples
- Increase the speed of learning processes by limiting the necessary experiments

Building more generalized models:

- learning to solve many tasks, not just one task: meta-learning does not focus on training one model on one specific data set

Optimization of Meta-learning

A hyperparameter is a parameter whose value guides the learning process. It is a parameter that is defined before the learning process begins. Hyper parameters have a direct effect on the quality of the training process. Hyper parameters can be tuned. A representative case of a hyperparameter is the number of branches in the decision tree. Many machine learning models have many hyper parameters that can be optimized. We mentioned that hyper parameters have a great influence on the training process. The hyperparameter selection process dramatically affects how well the algorithm learns. However, a challenge arises with the ever-increasing complexity of models or neural networks. Because of the complexity of the models, they are increasingly difficult to configure. Consider a neural network. Human engineers can optimize several parameters for the configuration. This is done through experimentation. Yet deep neural networks

have a various number of hyper parameters. Such a system has become too complicated for humans to optimize fully. There are many ways to optimize hyper parameters.

Grid Search: This method uses manually predefined hyper parameters. A group of predefined parameters is searched for the most powerful. Grid search involves trying all possible combinations of hyperparameter values. The model then decides on the most appropriate hyperparameter value. However, this method is considered traditional because it is time-consuming and inefficient.

Random Search: Grid search is an exhaustive method. It involves binding all possible combinations of values. This method takes the place of this exhaustive process with a random search. The model creates random combinations and tries to fit the data set to test the accuracy. Since the search is random, there is a possibility that the model will miss several potentially optimal combinations. On the other hand, it uses much less time than grid search and often provides the ideal solution. A random search can outperform a grid search. This is under the condition that several hyper parameters are needed to optimize the algorithm. We will cover these two and other optimization methods in another article. However, if you want to learn more about grid search and random search now, check out this hyperparameter tuning conceptual guide.